

Software Design

What is Software Design?

- Software design is the process of transforming requirements and specifications into a blueprint for developing software.
- It defines the architecture, components, modules, interfaces, and data flow, ensuring the system meets functional and non-functional requirements.
- Good software design provides clarity, structure, and guidance for implementation, ensuring that the software is reliable, maintainable, and scalable.

Activities in the Design Phase

The **design phase** bridges the gap between requirements analysis and software development. Below are the key activities performed during this phase:

- 1. System Design (High-Level Design):** Define the overall architecture of the system, including components and their interactions.
 - Identifying system components, subsystems, and external interfaces.
 - Output: **Architecture Diagram**: Shows how the system is divided into components.
 - Example: A multi-tier architecture for a web application (client, server, and database).
- 2. Detailed Design (Low-Level Design):** Provide detailed specifications of individual components and their functionality. Activities:
 - Describing how each module or function will perform tasks.
 - Output: **Class Diagrams** for object-oriented design. **Flowcharts** or Pseudo-code to describe algorithms.
 - Example: A class diagram showing relationships between User, Order, and Payment classes.
- 3. Data Design:** Structure the data storage and define how data flows through the system. Activities:
 - Designing **database schemas** (tables, keys, constraints).
 - Specifying data access strategies.
 - Example: ER diagrams showing relationships between entities like Customer, Product, and Order.
- 4. Interface Design:** Define the user interfaces (UI) and system interfaces. Activities:
 - Designing screens, reports, and forms that users interact with.
 - Defining **API specifications** for system-to-system communication.
 - Example: Wireframes for a web application's login screen.
- 5. Component Design and Integration Plan:** Identify and design components/modules to be developed. Activities:
 - Breaking the system into small, manageable components.
 - Planning integration points between components.
 - Example: A microservice design where each service handles a specific function, such as authentication.

Software Design

6. **Security Design:** Integrate security measures in the software architecture. Activities:
- Identifying potential threats and vulnerabilities.
 - Designing **authentication, encryption, and access control** mechanisms.
 - Example: Implementing role-based access control (RBAC) for an enterprise system.

High-Level Design (HLD)

- ✓ High-level design is a system design. A High-level Design is a simple plan with a bird's eye view that shows everyone involved in the project what the final system will look like after it's built and ready to use. Non-technical people can understand the architecture and design of the final solution using HLD.
- ✓ The participants in a high-level design are the client team, review team, and design team.
- ✓ HLD (High-Level Design) gives an overview of how the entire system is structured, with high-level details about the system architecture, components, and major interactions.
- ✓ In the Library Management System: HLD includes defining the modules (User, Admin, Database), choosing the technology stack, and describing major workflows (search, borrow, return books).

Low Level Design (LLD)

- ✓ Commonly abbreviated as LLD, the low-level design basically contains a detailed description of every module. In simpler words, the LLD describes every module in detail by including the actual logic of every component in the system. It goes deeply into every system specification and is thus known as detailed/ micro-level design.
- ✓ The participants in a low-level design are the operation teams, design team, and implementers.
- ✓ In the Library Management System: LLD specifies individual classes (Book, User, Transaction), methods for interacting with the database, API endpoints, and detailed database schema.

Comparison of High-Level Design (HLD) and Low-Level Design (LLD):

Here is a comparison of High-Level Design (HLD) and Low-Level Design (LLD) in a table format:

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Definition	Focuses on the system architecture and major components.	Focuses on the detailed design of individual components and modules.
Scope	Broad overview of the system and its components.	Detailed design of specific modules, components, and data flows.
Focus	Defines major system components and their interactions.	Defines the internal working of components, classes, and methods.

Software Design

Aspect	High-Level Design (HLD)	Low-Level Design (LLD)
Components Covered	System architecture, modules, sub-systems, data flow.	Algorithms, class diagrams, function signatures, database schema.
Input	Requirements, business goals, system constraints.	HLD, system specifications, technical requirements.
Output	System architecture diagram, module interaction diagram.	Class diagrams, sequence diagrams, database schema, API specifications.
Tools Used	Architecture diagrams, flowcharts, component diagrams.	UML diagrams, ER diagrams, pseudocode, detailed sequence diagrams.
Time of Creation	Created during the early stages of design (after requirements gathering).	Created after HLD, in the detailed design phase.

Characteristics of a Good Design

Below are the key characteristics of a good design, along with brief examples for better understanding:

- 1. User-Centered:** A good design prioritizes the needs, behaviors, and preferences of its users.
Example: A mobile app with an intuitive interface that minimizes user effort through simple navigation.
- 2. Functionality:** It must serve the intended purpose effectively and solve the problem it was designed for.
Example: A water bottle with a built-in filter ensures safe drinking water on the go.
- 3. Simplicity (Less is More):** Simple and minimal design reduces cognitive load and makes the product easy to understand and use.
Example: Google's homepage design is minimal, focusing on the search bar as the central element.
- 4. Aesthetic Appeal:** Visually pleasing designs attract users and create positive emotional connections.
Example: Apple's product designs combine sleek forms with elegance, enhancing user desirability.
- 5. Accessibility:** Inclusive design ensures that products and services are usable by people of all abilities.
Example: Websites with screen-reader support and accessible navigation enhance usability for visually impaired users.
- 6. Scalability and Flexibility:** A good design should adapt to future changes, upgrades, or needs.
Example: A modular office desk that can be rearranged or expanded according to space requirements.
- 7. Emotional Connection:** A good design evokes positive emotions, making the user feel good about interacting with it.
Example: The Volkswagen Beetle's design has historically evoked nostalgia and emotional connection.

Scenario: Library Management System

The Library Management System is a basic system that allows users (students and librarians) to manage book borrowing and returns. In this system, the main activities include Registration, login searching for books, borrowing books, and returning books. The system also lets librarians manage book inventory. And both can Logout.

1. Use Case Diagram

The primary actors in this scenario are:

- **Student:** Can self-register, log in, search for books, borrow books, return books and logout.
- **Librarian:** Can login, manage book inventory, assist with book borrowing and returning and logout.

Use Cases:

- **Register:** A user registers themselves in the system.
- **Login:** Both students and librarians can log in to the system.
- **Search Book:** Any registered user can search for a book.
- **Borrow Book:** Users can borrow books if available.
- **Return Book:** Users can return borrowed books.
- **Manage Book Inventory:** Librarians can add, update, and remove books from the system.
- **Logout:** Both students and librarians can log out of the system.

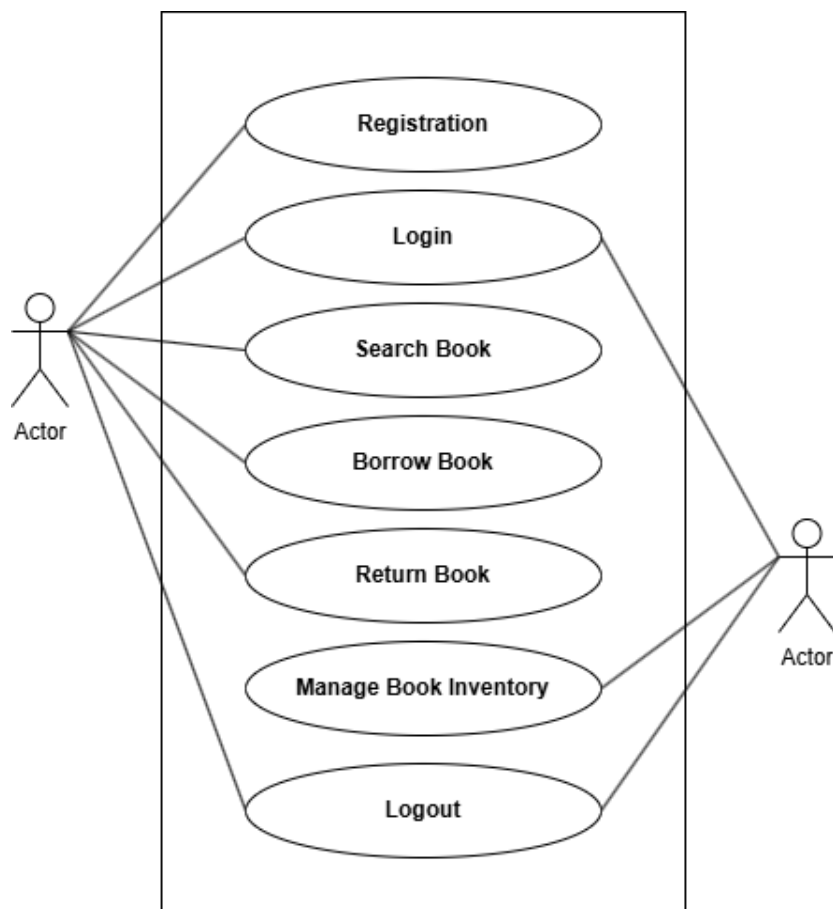


Figure-: Use Case Diagram for Library Management System

User Case Description

Case Description-01: Registration

Use Case	Registration	
Goal	Student must be logged in.	
Precondition	Users must have access for registration.	
Success End Condition	Notification: “Book Borrowed Successfully.”	
Failed End Condition	Notification: “Book is Unavailable.”	
Primary Actors:	Student	
Secondary Actors:		
Trigger	Student initiates a borrowing request for a specific book.	
Description / Main Success Scenario	1.	Student initiates a borrowing request for a specific book.
	2.	System checks book availability
	3.	Enter Information
	4.	If available, system registers the book as borrowed
	5.	System confirms the borrowing
	6.	The system saves the details and shows them !!!Successfully Booked!!! Notify
Alternative Flows	1.1	System Error
		1.1.a. Try Again!!
	2.1	System Doesn't work.
		2.1.a. Try Again Later!
	4.1	The user Did not fill up the details!
		4.1.a. Checked By the system & Notify by “Please! Fill Up the Box”.
	5.1	The system did not respond
		5.1.a. Show Error Message.
	6.1	The system Doesn’t save the details.
		6.1.a. Notification: “Details did not Save”
Quality Requirements	The user Will fill up all the details in 30 minutes.	

Case Description-02: Login

Use Case	Login	
Goal	Users can enter the system by the login.	
Precondition	Users must have to be registered First.	
Success End Condition	Notification: “Login Successful”	
Failed End Condition	Notification: “Login Failed!!”	
Primary Actors:	Student, Librarian	
Secondary Actors:		
Trigger	The user will request a login to enter the system	
Description / Main Success Scenario	1.	Press “Login” Button
	2.	Provide login interface
	3.	Enter user id and password.
	4.	User provide the “Login” button
	5.	Verified and login
	6.	Notification: “Login Successful”
Alternative Flows	1.1	System Error
		1.1.a. Try again
	2.1	Server Not Found
		2.1.a. Try Again Later!
	4.1	The system Did not respond.
		4.1.a. Show error message.
	5.1	Information Error!!
		5.1.a. Notification: “Enter the right User ID and Password.”
Quality Requirements	Users fill up the login info within 10 minutes.	

Case Description-03: Search Book

Use Case	Search Book	
Goal	The user searches for books by title, author, or genre in the system.	
Precondition	Must visit Search Box.	
Success End Condition	Notification “Here is the list:”	
Failed End Condition	Notification “Sorry, Nothing Found!”.	
Primary Actors:	Student	
Secondary Actors:		
Trigger	Users will search for books by title, author, or genre in the system.	
Description / Main Success Scenario		
	1.	Visit Dashboard
	2.	Press “Search”
	3.	Provide Search Box
	4.	Fill Search Box
	5.	User press “Search” button
	6.	Search Result shown as list.
Alternative Flows		
	2.1	System Error
		2.1.a Try again
	3.1	Not responding
		3.1.a Try later
	5.1	Not Found
		5.1.a Notification: “Sorry, Nothing Found!”
Quality Requirements	Search will be done by keywords	

Case Description-04: Borrow Book

Use Case	Borrow Book	
Goal	The student borrows a book if it is available.	
Precondition	Must visit Room Booking Option in Dashboard.	
Success End Condition	Notification “Room Booked Successfully”	
Failed End Condition	Notification “Room Not Booked Yet”.	
Primary Actors:	Customer, Manager	
Secondary Actors:		
Trigger	Users will request to book an empty room.	
Description / Main Success Scenario		
	1.	Visit Dashboard.
	2.	Select “Room Booking”
	3.	Select “Room Number”
	4.	Check room Available
	5.	Enter “Booking Date”.
	6.	Enter “Check Out Date”
	7.	Provide Customer information form.
	8.	Fill the information field.
	9.	Then press “Submit” button
	10.	Information saved
	11.	Notification: “Room booking Successful”
Alternative Flows		
	2.1	System did not respond.
		2.1.a. Try again
	3.1	System error
		3.1.a. Try again Later.
	4.1	Not Available room
		4.1.a Select another room
	5.1	Select date
		5.1.a. Must be added date
	6.1	Select date
		6.1.a. Must be added date
	7.1	Check empty field
		7.1.a. Fill up required field
	10.1	Information did not save
		10.1.a. Try again
Quality Requirements	Users will get 20 mins to fill up	

Case Description-05: Return Book

Use Case	Return Book	
Goal	The student returns a previously borrowed book.	
Precondition	The student must be logged in and have borrowed books in their account.	
Success End Condition	Book is successfully returned, and inventory is updated.	
Failed End Condition	Notification: "Return failed, please try again."	
Primary Actors:	Student	
Secondary Actors:		
Trigger	Users will request to checkout their booked room	
Description / Main Success Scenario		
	1.	Visit Dashboard.
	2.	Student navigates to the Borrowed Books section.
	3.	Student selects the book to return.
	4.	Student confirms return action.
	5.	System updates inventory and removes the book from Student's account.
	6.	Notification: "Return successful."
	2.1	The system did not respond.
		2.1.a. Try again.
	4.1	Book Not Found in Account
		4.1.a. Notification: "Book not found in your account."
Quality Requirements	Users will get 20 mins to return	

Case Description-06: Manage Book Inventory

Use Case	Manage Book Inventory	
Goal	The librarian manages the library's inventory by adding, updating, or removing books.	
Precondition	Must be logged in this system.	
Success End Condition	Book inventory is successfully updated, reflecting additions, updates, or removals.	
Failed End Condition	Notification: "Inventory update failed, please try again."	
Primary Actors:	Librarian	
Secondary Actors:		
Trigger	Librarian initiates an inventory action (add, update, or remove a book).	
Description / Main Success Scenario	1.	Visit Dashboard.
	2.	Librarian navigates to the Book Inventory section.
	3.	Librarian selects an action: Add, Update, or Remove Book.
	4.	Librarian provides required details for the selected action.
	5.	System updates the inventory based on the action taken.
Alternative Flows	2.1	Server Down.
		2.1.a. Try again later.
	3.1	Book Not Found for Update or Remove:
		3.1.a. Notification: "Book not found in inventory."
Quality Requirements	Inventory updates should reflect in real-time to ensure accurate availability status.	

Case Description-7: Logout

Use Case	Logout	
Goal	Users can Log Out from the system.	
Precondition	The user must log in to the system.	
Success End Condition	Notification: “Logout Successful”.	
Failed End Condition	Notification: “System Error”.	
Primary Actors:	Student, Librarian	
Secondary Actors:		
Trigger	The user will request a Logout	
Description / Main Success Scenario	1.	Log in to the system.
	2.	Select the “Logout” Option.
	3.	Press the “Logout” Button.
	4.	Logout successful
Alternative Flows		
	1.1	Server down
		1.1.a try again
	2.1	Not Responding
		2.1.a Try again
	3.1	Not Responding
		3.1.a Reload
Quality Requirements	The user will Immediately Logout After hitting the Logout Button.	

2. Activity Diagram: Borrow Book

The activity diagram below illustrates the process of a student borrowing a book:

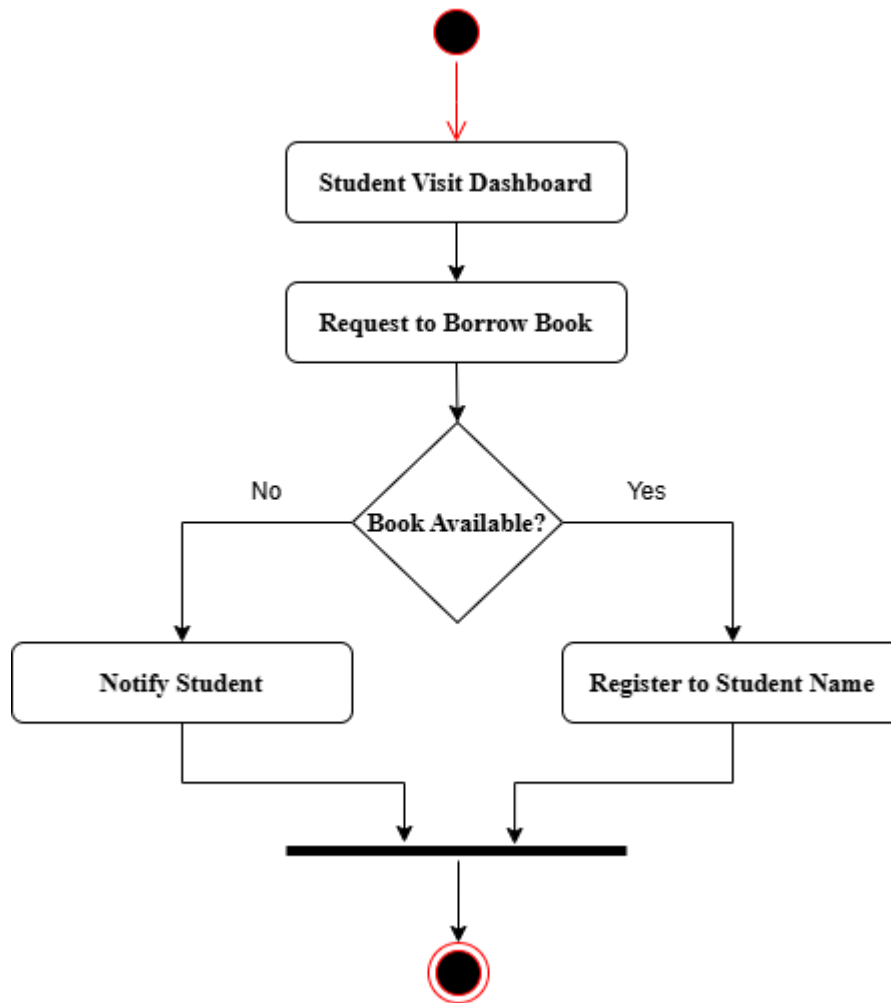


Figure-: Activity Diagram for Borrow Book Scenario

3. Sequence Diagram: Borrow Book

In this sequence diagram, the interactions between **Student**, **System**, and **Book Inventory** are shown for the "Borrow Book" process:

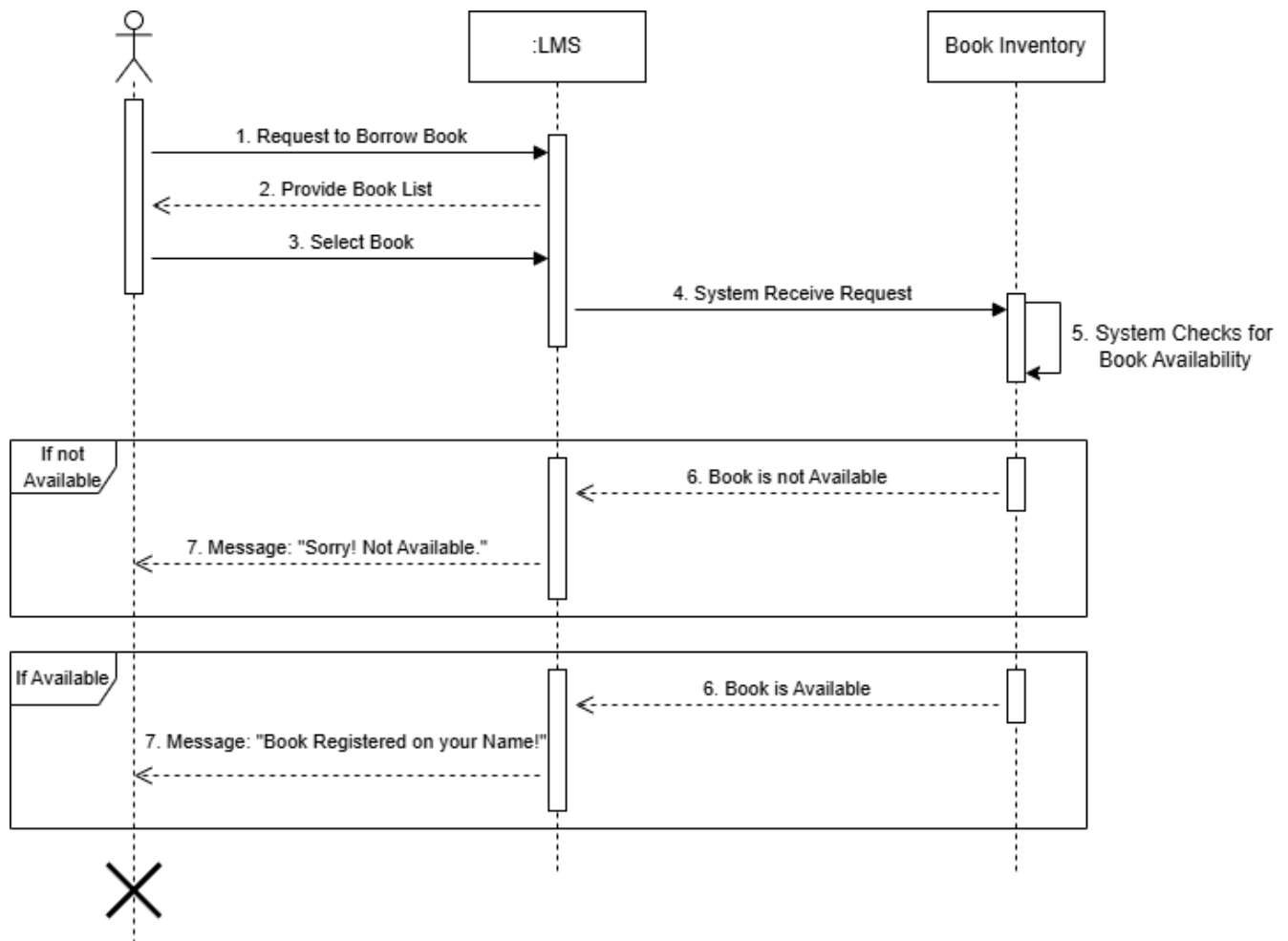


Figure-: Sequence Diagram for Borrow Book Scenario